

Everything about running NovaPay

A full-day, in-person workshop teaching UK students to use GitHub Copilot by debugging and extending a fake banking app. Students spend the morning fixing planted bugs, then the afternoon building their own feature from scratch — both guided by **Nova**, a customised Copilot Chat persona.

Y9–Y13 · ages 13–18

9:30am–2:30pm · 5 hours

1 dev per table · Mark MCs the room

~46 files in the pack

WHAT THIS BRIEFING DOESN'T COVER YET

Still open: who runs the lunchtime accessibility activities, expected student numbers, wifi capacity for the room (Codespaces need a stable connection per device), and any safeguarding/duty-of-care arrangements for the day. Worth nailing these down separately before running this again.

02 Logistics & Setup

Every student uses their own **personal GitHub account** — not an organisation account, since Business self-serve signup was paused when this was built. Each creates a repo from a template, then opens it in a **GitHub Codespace** — a cloud dev environment that works on any device, including school machines and iPads.

WHO RUNS WHAT

Mark MCs the whole room — clock, slides, phase transitions. **One developer owns each small table of students for the entire day.**

LIVE PREVIEW

VS Code Live Server — extension ID `ritwickdey.LiveServer`

AI MENTOR

Nova = Copilot Chat + custom instructions file

FILE ACCESS

Each developer works from local copies of the file pack on their own laptop

THE TWO-REPO SPLIT

Student's personal repo: only what they need — `bank-dashboard.html`, `starter-template.html`, `challenge.js`, devcontainer config, Nova's instructions.

Hub/reference pages: tutorials, screens, facilitator materials — browsed separately, never uploaded into student repos. Not hosted anywhere yet (open locally on your machine/projector).

03 The Day, Start to Finish

9:30am start, 5 hours, ends 2:30pm. Click any segment for the reasoning behind where it sits.

●	9:30	Welcome	15 min ▼
		Intros, Codespace setup, ice breaker. Longer than a typical warm-up because getting every account and Codespace genuinely working takes real time.	
●	9:45	Copilot demo	15 min ▼
		Live demo of inline completions and Chat — you drive, they watch, before anyone touches Copilot themselves.	
●	10:00	Bug tutorial — everyone together	10 min ▼
		Everyone follows a worked, deliberately fake example bug together, learning the method before touching the real dashboard.	
●	10:10	Independent bug hunt	20 min ▼
		Same method, applied to the real 10 planted bugs in bank-dashboard.html.	
●	10:30	Share out	5 min ▼
		2–3 students share what they found and how Copilot helped.	
●	10:35	Level Up ⚡	30 min ▼
		The reveal, GitHub/Codespaces concepts, and 3 challenge rounds: Fix the Bug → Comment Race → Best Prompt.	
●	11:05	☕ Comfort break	15 min ▼
		Genuinely just a break. Good moment for you to reset too before the longer build block ahead.	
●	11:20	Feature tutorial — everyone together	20 min ▼
		Everyone builds a Savings Tracker from a blank file, together — teaches the technique before anyone picks their own idea. Guided and low-stakes; help stragglers keep pace rather than fall behind, since it's meant to be followed as a group.	
●	11:40	Brainstorm	15 min ▼

Their own idea, deliberately different from the tracker they just built together. Facilitator role matters here: check every group's pick against 'can you describe it in one sentence — inputs, calculation, output?', and redirect anything needing logins/databases toward a one-screen version instead.

● **11:55 Plan on paper** 10 min ▼

Four questions, no keyboards: what it does, what the user sees, inputs, outputs. The one thing worth checking on every sketch: are the inputs and outputs actually named? Vague sketches lead to vague, unproductive Copilot prompts later.

● **12:05pm Lunch** 30 min ▼

Accessibility activities happen during this window. Sits before any feature creation starts on purpose — nobody's mid-flow when they break.

● **12:35pm Accessibility discussion** 5 min ▼

Quick group chat incorporating what came up at lunch, right before building starts — so any idea has somewhere to go.

● **12:40pm Feature creation** 85 min ▼

One continuous block: structure, logic, styling, and testing all folded in. This is the single biggest, most facilitation-dependent phase of the whole day — see the dedicated section below for what good guidance looks like here.

● **2:05pm Show & tell** 15 min ▼

2-minute demos each: what it does, how Copilot helped, what they'd add with more time. If a demo breaks live, reframe it as 'the most authentic developer experience of the day' and show it working in the editor instead.

● **2:20pm Feedback** 10 min ▼

Quick round before everyone leaves — keep it short, energy is naturally dropping.

04 How Nova Works

Nova is just GitHub Copilot Chat with a custom `.github/copilot-instructions.md` file in each student's repo. It behaves differently depending on which phase of the day it is.

— PHASE-PANEL RESTRICTIVE ACTIVE —

During the bug hunt, Nova gives **hints, not answers**. It won't just fix the bug for a student who pastes in broken code.

Verify it's working: check the **References list** on a Copilot Chat response — click it to confirm the instructions file was actually used. More reliable than judging by tone alone.

— PHASE-PANEL GENERATIVE —

During feature creation, Nova helps build — but **Agent mode** (distinct from Ask/Chat) is action-biased, so it needs a hard rule: make one real fix, then stop and explain.

The first version of that rule caused Nova to *refuse* fixes outright — overcorrected. It was rewritten to require action, just paced action.

IF ONE STUDENT'S NOVA IS TOO RESTRICTIVE

It only affects that student — each has their own repo. Rename their `.github/copilot-instructions.md` on github.com, rebuild their Codespace, and Copilot reverts to normal for them only.

05 Guiding Feature Creation

This is the part earlier drafts of this briefing skated over — and it's the single biggest, most facilitation-dependent chunk of the day. Bug Hunt and Level Up mostly run themselves off the slide decks and Nova's restrictive instructions. Everything from Brainstorm onward runs on how well a developer facilitates it.

BRAINSTORM

Landing a workable idea ▲

THE ACHIEVABILITY CHECK

For every group's pick, ask: *"Can you describe it in one sentence? Does it have inputs, a calculation, and an output?"* If they're struggling to answer, the idea isn't scoped yet — narrow it before they move on.

SCOPE VETO PHRASE

"Brilliant idea for version 2. What's the one-screen version of it you could demo today?"

Use it on anything needing logins, real accounts, a database, or "like the real app but..."

IF A GROUP IS STUCK

Point at the back-pocket gallery on brainstorm-screen.html slide 2 — 7 pre-vetted features with difficulty labels. Just not the Savings Tracker, they've already built that one.

PLAN

The one thing worth checking ▲

Circulate and look at every sketch. The single check that matters: **are the inputs and outputs named?** ("goalName, targetAmount" — not "some boxes"). Named inputs become good Copilot prompts a few minutes later; vague ones lead to vague, unproductive prompting during the build.

THE METHOD — SAY IT EARLY, REFER BACK TO IT CONSTANTLY

"One: write a comment describing the next small thing. Two: let Copilot suggest, press Tab. Three: save, check Live Server. Four: if it's wrong, ask Nova why. Small steps — comment, code, check."

ROUGH INTERNAL PACING (NOT ANNOUNCED OR ENFORCED, JUST A SENSE OF WHERE GROUPS SHOULD BE)

- ~25 min in: heading, inputs, and a button exist, opens without errors
- ~60 min in: styled, basic JS working — real output on screen
- ~75 min in: shifting into testing

PROMPTING — GOOD VS VAGUE

-  "Write a function that takes goalAmount and savedAmount and returns the percentage complete"
-  "Make a savings tracker" — too vague to build on

THE HARD SCOPE CALL — SAY IT TO THE WHOLE ROOM, NOT JUST THE STUCK GROUP

"Strip it back. One input, one button, one number on screen. Get that working, then add back."

TESTING IS FOLDED IN, NOT A SEPARATE SLOT

Nudge groups proactively as they near the end: "Code freeze soon — try to break your own feature on purpose."
feature-test-plan.html has a ready-to-demo checklist if a group wants structure.

FAST FINISHERS

milestone-screen.html's right panel has stretch goals by year group — persisting to localStorage, multiple goals, weekly-saving calculations, a simulate-transfer button.

2 minutes per group. After each, ask the room "what did you like about that?" If a demo breaks live:

"Congratulations — a live demo failure. That's the most authentic developer experience of the day."

Then show it working in the editor instead — never let a broken demo become an awkward silence.

06 The 10 Planted Bugs

All in `bank-dashboard.html`. Bugs 3, 8, 9, 10 are logic/data errors rather than syntax errors — if a group asks Copilot to "fix everything at once," it will likely miss these. Worth using as a live teaching moment if it happens.

#	LEVEL	BUG
1	Easy	Missing closing <code></div></code> tag
2	Easy	Balance text colour matches background (invisible)
3	Easy	Coffee purchase shows as credit instead of debit
4	Medium	Input type is <code>text</code> instead of <code>number</code>
5	Medium	Button type: <code>onlick</code> instead of <code>onclick</code>
6	Medium	Wrong operator — adds instead of subtracts on withdrawal
7	Hard	Element ID typo (<code>balence-display</code> vs <code>balance-display</code>)
8	Hard	<code>NaN</code> shown on empty input — missing <code>isNaN()</code> check
9	Hard	Wrong interest formula — missing <code>/100</code>
10	Hard	No overdraft check

07 Known Technical Gotchas

Everything below came out of your own testing. Click to expand.

Codespace staleness

Creating a Codespace before adding a file to the repo means it won't see that file until rebuilt (Ctrl+Shift+P → Rebuild Container) or recreated.

Save vs push — the one that causes most confusion

Ctrl+S only saves to the Codespace's own disk — it does NOT push to GitHub. Only commit + push updates the repo. A new Codespace always pulls from GitHub's last-pushed state, so an unpushed fix is lost if that Codespace gets deleted.

If git push hangs

Bypass the Codespace entirely — paste the file content into github.com's own web editor and commit there instead. It uses the browser's own connection rather than whatever's blocking the Codespace's git path.

If Nova is too restrictive for one student

Only affects that student — each has their own repo. Rename their .github/copilot-instructions.md on github.com, rebuild their Codespace, and Copilot reverts to normal for them only.

challenge.js is pre-loaded, not created fresh

Asking Copilot Chat to review a cold-pasted loan-calculation snippet sometimes triggered a false 'SQL injection' refusal. Pre-loading it as a real repo file avoids that entirely.

08 File Map

All in `/mnt/user-data/outputs/`. Selected highlights — see the pack for the full ~46.

START HERE

`index.html`
`run-the-session.html`
`admin-setup.html`
`student-index.html`

AUTHORITATIVE TIMING DOCS

`session-plan.html`
`printable-master-script.html`
`printable-timed-agenda-clock.html`
`student-day-agenda.html`

SETUP GUIDES

`setup-guide-v2.html`
`printable-setup-guide-v2.html`
`facilitator-setup-guide.html`

STUDENT TUTORIALS

`how-to-find-fix-bugs.html`
`how-to-build-your-feature.html`
`participant-guide.html`

LEVEL UP

`level-up-screen.html`
`copilot-bootcamp.html`

OUT OF STEP — NEEDS A DECISION

`build-phase-guide.html`

09 Before You Run This Again

- ☐ Check the Copilot billing/limits situation for student accounts — the 50 chat-messages/month cap on personal Free accounts is the one that could actually break the day
- ☐ Do one fresh click-through of the real Copilot Chat / Agent interface in VS Code — the UI has moved on since this was built
- ☐ Re-verify: upload the current copilot-instructions.md + challenge.js to a test repo, confirm the References list shows the instructions file
- ☐ Confirm Agent mode makes one fix then stops, rather than refusing or fixing everything at once
- ☐ Print or open: Master Script, Timed Agenda (clock version), Troubleshooting Card, Roster, Preflight Checklist
- ☐ Decide what to do with build-phase-guide.html — fix, retire, or ignore